



University of
Nottingham
UK | CHINA | MALAYSIA

Lecture 10: Markov Chain & Reinforcement Learning

COMP1037 Fundamentals of Artificial Intelligence

Xinan Chen

University of Nottingham Ningbo China

April 27, 2025

Outline

1. Introduction
2. RL Introduction
3. Markov Chains
4. RL Agent Design
5. Q-Learning
6. Deep Q-Learning

Train an AI to Play Computer Game

Question

How can we train an AI model to play and get high score at computer games (like Super Breakout in Atari 2600)?

Tasks:

1. Try to play this game.
2. Find a **more efficient way** to get high score.

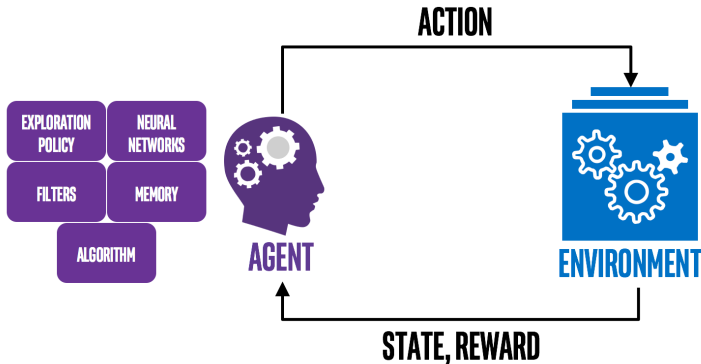
Hints:

1. Click the **right button** to reset game (F12)
2. Then, click **Space** to start

https://www.retrogames.cz/play_223-Atari2600.php



Reinforcement Learning (RL) in Breakout

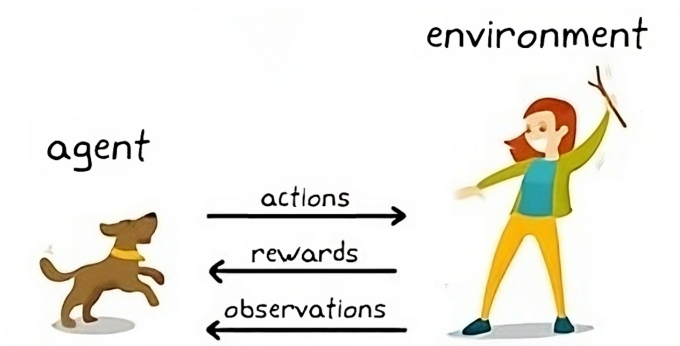


<https://www.youtube.com/watch?v=V1eYniJ0Rnk>

Reinforcement Learning (RL) in Breakout

What does the Q-Learning agent learn?

The agent realizes that **digging a tunnel through the wall** is the most effective technique to beat the game.



Choosing Reward Features for Breakout

Question

What **features should be considered** as **potential rewards/punishments** when training an AI to play the game Super Breakout?

Choosing Reward Features for Breakout

Question

What **features should be considered** as **potential rewards** when training an AI to play the game Super Breakout?

- **Score Got** - reward
- **Block Destruction** - reward
- **Time Used** - punishment
- **Lives Used** - punishment
- **Level Completion** - reward

Reward Engineering

Reward engineering is critical in reinforcement learning as it directly influences the agent's behavior by defining what it learns to optimize.

The Markov Property

Definition

The **Markov Property**, also known as **memorylessness**, is a fundamental property of a stochastic process where the **conditional probability distribution of future states of the process depends only upon the present state**, not on the sequence of events that preceded it.

Mathematical Formulation

In mathematical terms, for any time s, t with $s < t$, the Markov Property can be expressed as:

$$P(X_t = x | X_s = x_s, X_{s-1} \dots, X_0 = x_0) = P(X_t = x | X_s = x_s)$$

where X_t represents the state of the process at time t .

Markov Chains

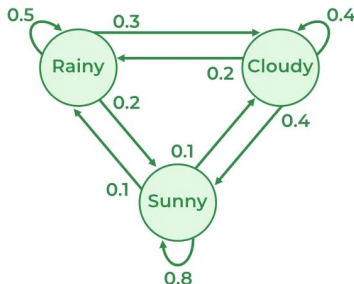
Definition

A **Markov Chain** is a stochastic model describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous event.

Characteristics

- **Memoryless:** The **future state depends only on the current state**, not on the sequence of events that preceded it.
- **State Space:** A set of all possible states in which the process could be.
- **Transition Probability:** Probability of moving from one state to another, typically represented in a matrix form.

Markov Chains - Example



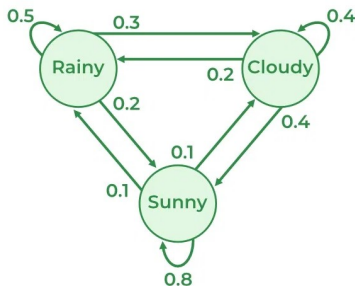
$$P = \begin{bmatrix} & R & C & S \\ R & 0.5 & 0.3 & 0.2 \\ C & 0.2 & 0.4 & 0.4 \\ S & 0.1 & 0.1 & 0.8 \end{bmatrix}$$

After 2 days:

$$P^2 = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.2 & 0.4 & 0.4 \\ 0.1 & 0.1 & 0.8 \end{bmatrix} \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.2 & 0.4 & 0.4 \\ 0.1 & 0.1 & 0.8 \end{bmatrix}$$

$$\begin{bmatrix} & R & C & S \\ R & 0.33 & 0.29 & 0.38 \\ C & 0.22 & 0.26 & 0.52 \\ S & 0.15 & 0.15 & 0.7 \end{bmatrix}$$

Markov Chains - Example - Stable State



$$P = \begin{bmatrix} & R & C & S \\ R & 0.5 & 0.3 & 0.2 \\ C & 0.2 & 0.4 & 0.4 \\ S & 0.1 & 0.1 & 0.8 \end{bmatrix}$$

After 10 days:

$$P^{10} = \begin{bmatrix} & R & C & S \\ R & 0.2 & 0.2 & 0.6 \\ C & 0.2 & 0.2 & 0.6 \\ S & 0.2 & 0.2 & 0.6 \end{bmatrix}$$

From Markov Chain to MDP

Markov Chain (MC)

- A stochastic process with **no control**
- Transitions happen *automatically* based on probability
- Defined by: (S, P) where S = states, P = transition matrix

Markov Decision Process (MDP)

- A stochastic process **with control** (decision making)
- Agent can *choose actions* to influence transitions
- Defined by: (S, A, P, R, γ) where:
 - ▶ S = states, A = actions
 - ▶ $P(s'|s, a)$ = transition probability
 - ▶ $R(s, a)$ = reward function
 - ▶ γ = discount factor

MDP to Design a RL Agent

Introduction to Reinforcement Learning

Reinforcement Learning involves an agent that **learns to behave in an environment by performing actions and seeing the results**, aiming to maximize the reward.

Core Components

- **Agent:** The learner or decision-maker.
- **Environment:** Where the agent operates.
- **Actions:** What the agent can do.
- **Reward:** Feedback from the environment to evaluate actions.
- **Policy:** The strategy the agent employs to determine the next actions based on the current state.

Q-Learning - RL

What is Q-Learning?

Q-Learning is an **off-policy** reinforcement learning algorithm that seeks to find the best action to take given the current state.

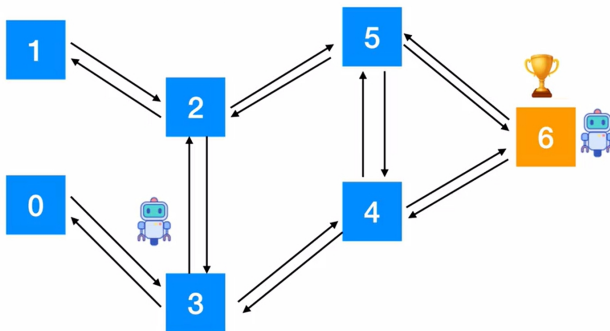
Q-Learning Algorithm

1. Initialize the Q-values table arbitrarily/zeros.
2. Observe the current state.
3. Select an action based on the current Q-values (e.g., greedy).
4. Execute the action and observe the reward and new state.
5. Update the Q-value using the formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

6. Repeat until the problem is solved or maximum iteration.

Q-Learning - Example



Problem

- Scenario: A robot is tasked with navigating from point 3 to point 6 in a given environment.
- Objective: Reach point 6 from point 3 while maximizing the accumulated score.

Q-Learning - Q-Table

	0	1	2	3	4	5	6
0	-1	-1	-1	1	-1	-1	-1
1	-1	-1	1	-1	-1	-1	-1
2	-1	1	-1	1	-1	1	-1
3	1	-1	1	-1	1	-1	-1
4	-1	-1	-1	1	-1	1	100
5	-1	-1	1	-1	1	-1	100
6	-1	-1	-1	-1	1	1	100

This is the reward table, but how to make the Q-table?

- A **Q-table** is a matrix-like data structure used in Q-learning.
- The Q-table helps in decision-making by providing a lookup table where the agent can quickly find the best action to perform in each state.

Q-Learning - Q-Table - Step 1

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

$$\forall Q(s, a) = 0$$

Q-Learning - Q-Table - Step 2

	0	1	2	3	4	5	6
0	-1	-1	-1	0	-1	-1	-1
1	-1	-1	0	-1	-1	-1	-1
2	-1	0	-1	0	-1	0	-1
3	0	-1	0	-1	0	-1	-1
4	-1	-1	-1	0	-1	0	0
5	-1	-1	0	-1	0	-1	0
6	-1	-1	-1	-1	0	0	-1

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

$$\text{unavailable } Q(s, a) = -1$$

Q-Learning - Q-Table - Step 3

	0	1	2	3	4	5	6
0	-1	-1	-1	0	-1	-1	-1
1	-1	-1	0	-1	-1	-1	-1
2	-1	0	-1	0	-1	0	-1
3	0	-1	0	-1	1	-1	-1
4	-1	-1	-1	0	-1	1	0
5	-1	-1	0	-1	0	-1	100
6	-1	-1	-1	-1	0	0	-1

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

Route $\rightarrow [3, 4, 5, 6]$

$$Q(3, 4) = 1 + 0.8 \cdot 0 = 1$$

$$Q(4, 5) = 1 + 0.8 \cdot 0 = 1$$

$$Q(5, 6) = 100 + 0.8 \cdot 0 = 100$$

Q-Learning - Q-Table - Step 4

	0	1	2	3	4	5	6
0	-1	-1	-1	0	-1	-1	-1
1	-1	-1	0	-1	-1	-1	-1
2	-1	0	-1	0	-1	0	-1
3	0	-1	0	-1	1.8	-1	-1
4	-1	-1	-1	0	-1	81	0
5	-1	-1	0	-1	0	-1	100
6	-1	-1	-1	-1	0	0	-1

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

Route $\rightarrow [3, 4, 5, 6]$

$$Q(3, 4) = 1 + 0.8 \cdot 1 = 1.8$$

$$Q(4, 5) = 1 + 0.8 \cdot 100 = 81$$

$$Q(5, 6) = 100 + 0.8 \cdot 0 = 100$$

Q-Learning - Q-Table - Step 5

	0	1	2	3	4	5	6
0	-1	-1	-1	0	-1	-1	-1
1	-1	-1	0	-1	-1	-1	-1
2	-1	0	-1	0	-1	0	-1
3	0	-1	0	-1	65.8	-1	-1
4	-1	-1	-1	0	-1	81	0
5	-1	-1	0	-1	0	-1	100
6	-1	-1	-1	-1	0	0	-1

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

Route $\rightarrow [3, 4, 5, 6]$

$$Q(3, 4) = 1 + 0.8 \cdot 81 = 65.8$$

$$Q(4, 5) = 1 + 0.8 \cdot 100 = 81$$

$$Q(5, 6) = 100 + 0.8 \cdot 0 = 100$$

Q-Learning - Q-Table - Question 1

	0	1	2	3	4	5	6
0	-1	-1	-1	0	-1	-1	-1
1	-1	-1	0	-1	-1	-1	-1
2	-1	0	-1	0	-1	0	-1
3	0	-1	0	-1	65.8	-1	-1
4	-1	-1	-1	0	-1	81	0
5	-1	-1	0	-1	0	-1	100
6	-1	-1	-1	-1	0	0	-1

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

Question Route $\rightarrow [3, 4, 6]$

Update the Q-table until the value does not change.

Q-Learning - Q-Table - Answer 1

	0	1	2	3	4	5	6
0	-1	-1	-1	0	-1	-1	-1
1	-1	-1	0	-1	-1	-1	-1
2	-1	0	-1	0	-1	0	-1
3	0	-1	0	-1	81	-1	-1
4	-1	-1	-1	0	-1	81	100
5	-1	-1	0	-1	0	-1	100
6	-1	-1	-1	-1	0	0	-1

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

Question Route $\rightarrow [3, 4, 6]$

$$Q(3, 4) = 1 + 0.8 \cdot 100 = 81$$

$$Q(4, 6) = 100 + 0.8 \cdot 0 = 100$$

Q-Learning - Q-Table - Question 2

	0	1	2	3	4	5	6
0	-1	-1	-1	0	-1	-1	-1
1	-1	-1	0	-1	-1	-1	-1
2	-1	0	-1	0	-1	0	-1
3	0	-1	0	-1	81	-1	-1
4	-1	-1	-1	0	-1	81	100
5	-1	-1	0	-1	0	-1	100
6	-1	-1	-1	-1	0	0	-1

$$\alpha = 1, \gamma = 0.8$$

$$Q(s, a) = R + 0.8 \max_{a'} Q(s', a')$$

Question

Update the whole Q-table value.

Q-Learning - Q-Table - Question 3

	0	1	2	3	4	5	6
0	-1	-1	-1	65.8	-1	-1	-1
1	-1	-1	65.8	-1	-1	-1	-1
2	-1	53.64	-1	65.8	-1	81	-1
3	53.64	-1	65.8	-1	81	-1	-1
4	-1	-1	-1	65.8	-1	81	100
5	-1	-1	65.8	-1	81	-1	100
6	-1	-1	-1	-1	-1	-1	-1

Question

According to the Q-table, write the route when starting at node 0.

Q-Learning - Q-Table - Answer 3

	0	1	2	3	4	5	6
0	-1	-1	-1	65.8	-1	-1	-1
1	-1	-1	65.8	-1	-1	-1	-1
2	-1	53.64	-1	65.8	-1	81	-1
3	53.64	-1	65.8	-1	81	-1	-1
4	-1	-1	-1	65.8	-1	81	100
5	-1	-1	65.8	-1	81	-1	100
6	-1	-1	-1	-1	-1	-1	-1

Answer

$[0 \rightarrow 3 \rightarrow 4 \rightarrow 6]$

Q-Learning Pseudo-Code

Initialize:

- Initialize the Q-table $Q(s, a)$ arbitrarily/Zeros for all state-action pairs.
- Set the learning rate α and discount factor γ .

For each episode:

1. Initialize state s .
2. **While** state s is not terminal **do**:
 - 2.1 Choose action a from state s using policy derived from Q (e.g., ϵ -greedy).
 - 2.2 Take action a , observe reward r and new state s' .
 - 2.3 Update Q-value:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- 2.4 $s \leftarrow s'$.

3. end while

Repeat.

Deep Q-Learning

What is Deep Q-Learning?

$$Q(s, a) \rightarrow Q_{W,b}(s, a)$$

Algorithm Process

1. Initialize replay memory capacity.
2. Initialize the action-value network with random weights.
3. For each episode:
 - ▶ Initialize the sequence and the initial state.
 - ▶ For each step of the episode:
 - 3.1 Select an action via the policy derived from the Q-network.
 - 3.2 Execute the action and observe the reward and next state.
 - 3.3 **Compute the loss and update the network parameters.**
 - ▶ End the episode when a terminal state is reached.
4. Repeat for multiple episodes.

Loss Function in Deep Q-Networks (DQN)

Purpose of the Loss Function

The loss function in DQN aims to minimize the temporal-difference error, thereby converging predicted Q-values to the target Q-values.

Formulation of the Loss Function

$$L(W, b) = \left(r + \gamma \max_{a'} Q(s', a'; W^-, b^-) - Q(s, a; W, b) \right)^2$$

Parameter Update Mechanism

Parameters are updated via gradient descent to reduce the loss:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$$

Exercise

This grid world MDP operates as shown in Fig a. The states are grid squares, identified by their row and column number (row first). The agent marked as S. There are two terminal goal states, (2,3) with reward +5 and (1,3) with reward -5. Rewards are 0 in non-terminal states. The transition function is such that the intended agent movement (Up, Down, Left, or Right).

		+5
S		-5

(a)

Figure 1: (a) Gridworld MDP.

Exercise

Question:

Write the Q-value of this problem ($\alpha = 1, \gamma = 0.8$, unavailable node=-1) for the state (2,2) in **up, down, right, and left** actions.